Agendamento Rede Krill

Teste técnico para desenvolvedor · Candidato: **João Pedro** (GitHub JP-Legend) · Entrega em 04/11/2026

Repositório: https://github.com/JP-Legend/desafio-krill-agendamento
Vídeo de demonstração (14 min 01 s): página da release https://github.com/JP-Legend/desafio-krill-agendamento/releases/tag/v1.0.0
Link direto do vídeo: https://github.com/JP-Legend/desafio-krill-agendamento/releases/download/v1.0.0/video-demonstracao-agendamento-rede-krill.mp4

O que é

Sistema web para agendar a entrega de caminhões no recebimento do CD, com controle de capacidade de paletes por dia (480) e por faixa horária (120 em 4 faixas de 2 h). Não paletizado conta em dobro. A capacidade excedida gera alerta, sem bloquear. Ao agendar, o fornecedor recebe e-mail de confirmação. Há relatório por período com CSV, perfis com permissões validadas no backend e auditoria de todas as ações.

Como rodar localmente

Pré-requisito: Docker Desktop (ou Docker com o plugin Compose). Não é preciso instalar Java, Node nem PostgreSQL.

```
git clone https://github.com/JP-Legend/desafio-krill-agendamento.git
cd desafio-krill-agendamento
docker compose up -d --build
docker compose ps        # aguardar os 4 serviços como "healthy"
```

Serviço	Endereço
Aplicação (web)	http://localhost:8081
API	http://localhost:8080/api/v1
Swagger	http://localhost:8080/swagger-ui/index.html
E-mails enviados (Mailpit)	http://localhost:8025

Usuários padrão (Administrador e Consulta): veja a seção **“Usuários padrão”** do README. Para parar: `docker compose down`. Testes: `cd backend && mvn verify` e `cd frontend && npm run test:coverage`.

Stack

Backend	Java 17+, Spring Boot 3.5 (Web, Validation, Data JPA, Security com JWT), Flyway (V1 a V16), springdoc/Swagger, SMTP, Web Push
Frontend	Vue 3, Vite, PrimeVue, PWA
Banco	PostgreSQL 16
Infra	Dockerfiles multi-stage sem root, nginx, docker-compose com db, api, web e Mailpit
Testes	JUnit 5, Mockito, Testcontainers (Postgres real), JaCoCo; Vitest e Vue Test Utils

Banco de dados: por que PostgreSQL

- **Integridade:** agendamento, faixa, caminhão, paletização, CD, loja, fornecedor e auditoria são relacionais. As migrações Flyway criam chaves estrangeiras, `UNIQUE` e `CHECK` no próprio banco.
- **Agendamentos simultâneos:** cada agendamento roda numa transação que pega `pg_advisory_xact_lock` por data. Dois usuários no mesmo dia são atendidos em fila e a soma de paletes do alerta não fica errada.
- **Ocupação:** o agendamento guarda os paletes equivalentes; a % do dia e de cada faixa é calculada no domínio (`CalculadoraOcupacao`) a partir de uma consulta indexada por data.
- **JSONB** no "antes/depois" da auditoria e **índices parciais** nos tokens públicos.
- **Flyway** versiona o esquema; open source; roda na imagem oficial no compose e nos testes via Testcontainers (sem H2).

Decisões de arquitetura

- **Clean Architecture:** `domain` (regras puras, sem Spring) → `application` (casos de uso e portas) → `infrastructure` (JPA/JDBC, SMTP, push, JWT, BCrypt, ERP) e `web` (REST). Os casos de uso são montados em `UseCaseConfig` e testados com Mockito sem subir o Spring.
- **Frontend e backend separados**, conversando só por REST/JSON em `/api/v1` com JWT. Permissões decididas no backend.
- **Alerta em vez de bloqueio:** a calculadora devolve os alertas; o caso de uso salva e repassa para tela, e-mail e push.
- **Mailpit** como SMTP de desenvolvimento: os e-mails ficam visíveis em localhost:8025 e nada sai para fora. Envio assíncrono.
- **Docker multi-stage:** imagens finais só com JRE Alpine e nginx sem root, camadas do Spring Boot separadas para cache.

Detalhes: `docs/DECISOES-ARQUITETURA.md` no repositório.

Testes e cobertura (02/11/2026)

Parte	Testes	Linhas	Ramos
Backend: `domain`	119 (JUnit 5, Mockito, Testcontainers)	99,9%	90,6%
Backend: `application` (casos de uso)		97,0%	80,4%
Backend: total do projeto		93,7%	77,8%
Frontend: telas e módulos críticos	105 (Vitest)	91,1%	80,4%

Domínios críticos: `Agendamento` 100% de linhas, `CalculadoraOcupacao` 100%, `AgendamentoUseCase` 94,3%, `CapacidadeRecebimentoUseCase` 100%, `AcessoUseCase` (permissões) 98,1%, `RegrasAcesso` 100%, `AuditoriaUseCase` 100%. O `mvn verify` falha se domain + application ficarem abaixo de 80% de linhas (gate do JaCoCo), e o Vitest tem limite de 80%.

Relatório de segurança (resumo)

Ferramentas executadas em 02/11/2026: OSV-Scanner 2.7.0 (Maven e npm), Trivy 0.76.1 (imagens Docker), npm audit e Semgrep 1.182.1 (regras Java, OWASP Top 10, JavaScript e segredos). O plugin OWASP Dependency-Check está configurado no `pom.xml`; sem chave do NVD a base não baixa em tempo razoável, por isso a análise de dependências usou OSV-Scanner e Trivy.

Alvo	Antes	Depois

Dependências (OSV-Scanner)	22: 4 críticas, 2 altas, 16 médias	20: 4 críticas, 2 altas, 14 médias
Imagem api, jars (Trivy)	8: 2 críticas, 2 altas, 4 médias	6: 2 críticas, 2 altas, 2 médias
Imagem web e sistema da imagem api (Trivy)	0	0
npm audit só produção	0	0
Semgrep	2 avisos	2 avisos (falso positivo)

- **Corrigido:** commons-lang3 3.18.0, log4j-api 2.25.5 e o nginx passou a enviar o IP real em `X-Forwarded-For` (o IP da auditoria e do limite de requisições não pode ser forjado). Antes: Spring Boot 3.5.16 e bibliotecas atualizadas, imagem nginx atualizada.
- **Restante, justificado:** 2 críticas no spring-webmvc (só com XsltView ou SSE com fragmentos de view, que a API não usa; correção só no Spring 7), 2 altas e 2 médias no async-http-client (WebSocket/Kerberos, não usados pelo Web Push; correção só na 3.x), commons-compress só em teste, e pacotes do Vitest só em desenvolvimento (as 2 críticas restantes do OSV-Scanner são do tinypool).
- **Controles da aplicação:** BCrypt, JWT stateless, permissões numeradas validadas no backend (403), escopo por CD/loja nos casos de uso, auditoria com IP e antes/depois, limite por IP no link público (HTTP 429), Bean Validation, consultas parametrizadas, XML sem XXE, CORS restrito, cabeçalhos de segurança, segredos por variável de ambiente e containers sem root.

Relatório completo: `docs/RELATORIO-SEGURANCA.md`; saídas das ferramentas em `docs/seguranca/`.

Checklist do desafio

Requisito	Onde está
Capacidade 480/dia e 120/faixa, alerta sem bloqueio, não paletizado em dobro	`CalculadoraOcupacao`, `AgendamentoUseCase`, tela Novo agendamento
Cadastro de agendamento com % de ocupação	`AgendamentoController`, tela Novo agendamento
E-mail ao fornecedor	SMTP + Mailpit (localhost:8025)
Relatório por período + CSV	Tela Relatório, endpoint de exportação
Perfis e permissões no backend	`JwtFiltro`, `RegrasAcesso`, tela Perfis e permissões
Auditoria	`FiltroAuditoria`, `AuditoriaUseCase`, tela Auditoria
Documentação Swagger	localhost:8080/swagger-ui/index.html
Dockerfile otimizado e docker-compose	`backend/Dockerfile`, `frontend/Dockerfile`, `docker-compose.yml`
Testes com 80% ou mais nos domínios críticos	JaCoCo e Vitest (seção acima)
Histórico de commits coerente	Histórico do repositório no GitHub
Clean Architecture (diferencial)	Pacotes `domain`, `application`, `infrastructure`, `web`
README explicativo	`README.md`
Justificativa de banco e arquitetura	`docs/DECISOES-ARQUITETURA.md`
Relatório de segurança	`docs/RELATORIO-SEGURANCA.md`
Vídeo de até 15 minutos	Release v1.0.0 (14 min 01 s)